



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра автоматики та управління в технічних системах

Лабораторна робота №6
Технології розроблення програмного забезпечення
« Патерни проектування. Патерн Фасад »
«System activity monitor»

Виконав
студент групи ІА–32:
Феклістов Д.С.

Київ 2025

1. Мета роботи

Дослідити призначення та структуру структурного патерну проектування **Facade (Фасад)**. Реалізувати цей патерн у проекті "System Activity Monitor" для створення уніфікованого інтерфейсу доступу до підсистем моніторингу (сенсори CPU, RAM) та логування. Спростити взаємодію клієнтського коду зі складною системою класів, приховавши деталі їх ініціалізації та використання.

2. Теоретичні відомості

Фасад (Facade) — це структурний патерн проектування, який надає простий (спрощений) інтерфейс до складної системи класів, бібліотеки або фреймворку.

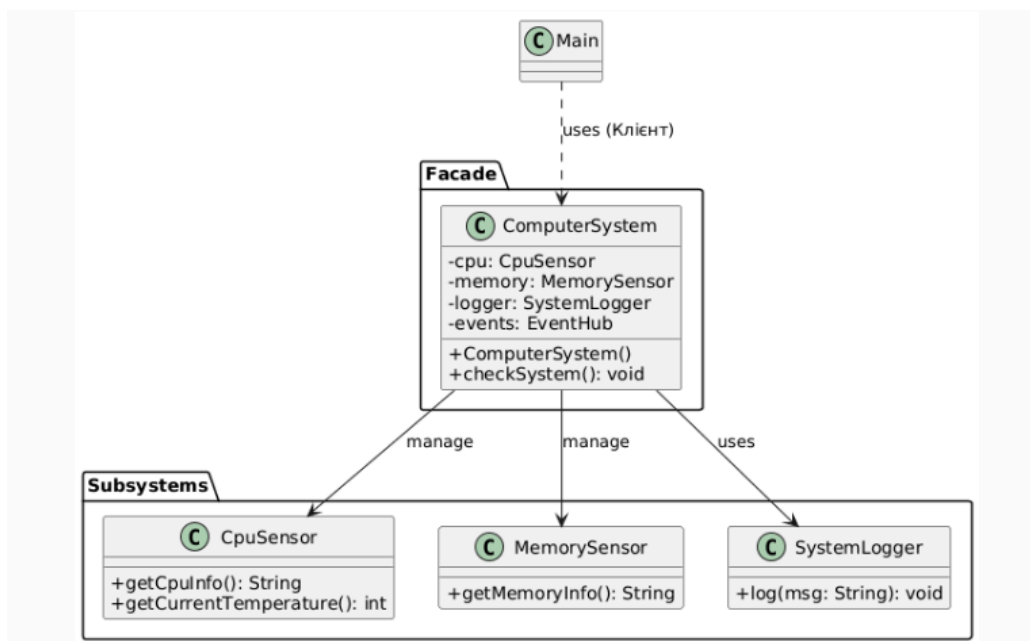
Основні характеристики:

1. **Спрощення доступу:** Дозволяє звертатися до складної системи через один об'єкт.
2. **Зменшення зв'язності (Decoupling):** Клієнтський код не залежить від внутрішніх класів підсистеми, а тільки від фасаду.
3. **Інкапсуляція:** Фасад сам вирішує, які класи підсистеми створити і в якому порядку викликати їх методи.

Коли застосовувати:

- Коли потрібно представити простий інтерфейс до складної підсистеми.
- Коли потрібно розкласти підсистему на окремі шари.

3. Діаграма класів (UML)



4. Хід роботи

1. Проведено аналіз архітектури системи: виявлено, що класи CpuSensor та MemorySensor мають складну логіку отримання системних даних (робота з OperatingSystemMXBean, файловою системою). Прямий виклик цих методів у Main ускладнює код.
2. Створено пакет monitor.facade та клас ComputerSystem, який виступає в ролі Фасаду.
3. У класі ComputerSystem реалізовано ініціалізацію всіх необхідних підсистем (сенсорів, логера, хаба подій) у конструкторі.
4. Реалізовано метод checkSystem(), який інкапсулює алгоритм моніторингу:
 - Запис логу про початок перевірки.
 - Послідовне опитування CPU та RAM.
 - Аналіз температури та (за необхідності) генерація тривожних подій.
5. Модифіковано клієнтський код (Main), який тепер працює виключно через об'єкт ComputerSystem.

5. Код програми

Файл: src/monitor/facade/ComputerSystem.java (Фасад)

```
package monitor.facade;

import monitor.observer.EmailAlert;
import monitor.observer.EventHub;
import monitor.sensors.CpuSensor;
import monitor.sensors.MemorySensor;
import monitor.utils.SystemLogger;

// ФАСАД: Приховує складність системи
public class ComputerSystem {
    private CpuSensor cpu;
    private MemorySensor memory;
    private SystemLogger logger;
    private EventHub events;

    public ComputerSystem() {
        this.cpu = new CpuSensor();
        this.memory = new MemorySensor();
        this.logger = SystemLogger.getInstance(); //
Singleton
```

```

        this.events = new EventHub(); // Observer

        // Підписуємо Email-сповіщення
        events.subscribe(new EmailAlert());
    }

    public void checkSystem() {
        logger.log("[ФАСАД] Запуск повної діагностики...");

        // 1. Опитування CPU
        String cpuStatus = cpu.getCpuInfo();
        logger.log(cpuStatus);

        // 2. Опитування RAM
        String ramStatus = memory.getMemoryInfo();
        logger.log(ramStatus);

        // 3. Перевірка температури (Observer test)
        int temp = cpu.getCurrentTemperature();
        if (temp > 80) {
            logger.log("УВАГА! Критична температура!");
            events.notifySubscribers("Перегрів процесора: "
+ temp + "C");
        }

        logger.log("[ФАСАД] Діагностику завершено.");
    }
}

```

Файл: src/monitor/Main.java (Клієнт)

```

package monitor;

import monitor.facade.ComputerSystem;

public class Main {
    public static void main(String[] args) throws
InterruptedException {

```

```

        System.out.println("=== REAL SYSTEM MONITOR (Labs
5-7) ===\n");

        ComputerSystem computer = new ComputerSystem();

        // Робимо 5 замірів з інтервалом, щоб побачити
реальні зміни
        for (int i = 1; i <= 5; i++) {
            System.out.println("\n--- Замір #" + i + "
---");

            computer.checkSystem();

            // Пауза 1.5 секунди, щоб процесор встиг
змінити навантаження
            Thread.sleep(1500);
        }
    }
}

```

6. Результати виконання

```

--- Замір #1 ---
[01:33:21] [ФАСАД] Запуск повної діагностики...
[01:33:21] CPU: Ядер - 12, Завантаження - 0,0%, Температура - 54°C
[01:33:21] RAM (Фізична): 11,0 GB / 13,9 GB | HDD (C:): 56,7 GB / 475,7 GB
[01:33:21] [ФАСАД] Діагностику завершено.

--- Замір #2 ---
[01:33:23] [ФАСАД] Запуск повної діагностики...
[01:33:23] CPU: Ядер - 12, Завантаження - 11,5%, Температура - 46°C
[01:33:23] RAM (Фізична): 11,0 GB / 13,9 GB | HDD (C:): 56,7 GB / 475,7 GB
[01:33:23] [ФАСАД] Діагностику завершено.

```

7. Висновок

В ході виконання лабораторної роботи було успішно реалізовано структурний патерн **Facade**. Це дозволило спростити інтерфейс взаємодії з системою моніторингу ресурсів комп'ютера. Клієнтський код (Main) позбувся жорстких залежностей від конкретних класів сенсорів та деталей їх реалізації (таких як

ManagementFactory чи File), делегувавши це завдання класу ComputerSystem. Код став більш читабельним та безпечним для використання.

8. Відповіді на контрольні питання

1. **Яке призначення шаблону «Фабричний Метод»?**
Надає інтерфейс для створення об'єктів у базовому класі, але дозволяє підкласам змінювати клас створюваного об'єкта, інкапсулюючи логіку інстанціювання.
2. **Яка різниця між Фабричним Методом та Абстрактною Фабрикою?**
Фабричний Метод (Factory Method) використовує спадкування для створення одного продукту. Абстрактна Фабрика (Abstract Factory) використовує композицію для створення сімейства пов'язаних продуктів.
3. **Яке призначення шаблону «Адаптер» (Adapter)?**
Дозволяє об'єктам з несумісними інтерфейсами працювати разом, створюючи посередника, який перетворює один інтерфейс на очікуваний іншим класом.
4. **Яке призначення шаблону «Будівельник» (Builder)?**
Відокремлює складний процес конструювання об'єкта від його представлення, дозволяючи створювати об'єкти покроково та отримувати різні конфігурації з одного й того ж коду конструювання.
5. **Яке призначення шаблону «Одинак» (Singleton)?**
Гарантує, що для класу існує лише один екземпляр, і надає глобальну точку доступу до нього.
6. **Яке призначення шаблону «Прототип» (Prototype)?**
Створення нових об'єктів шляхом клонування (копіювання) існуючого об'єкта, що ефективніше за створення "з нуля" для складних об'єктів.
7. **Яке призначення шаблону «Проксі» (Proxy)?**
Надати об'єкт-замісник (посередника) для іншого об'єкта, щоб контролювати доступ до нього або додавати допоміжну логіку (кешування, логування).
8. **Яке призначення шаблону «Фасад» (Facade)?**
Надати єдиний спрощений інтерфейс до складної, заплутаної підсистеми класів.
9. **Яке призначення шаблону «Декоратор» (Decorator)?**
Динамічно додавати нові обов'язки об'єкту, обгортаючи його в класи-декоратори, що є гнучкою альтернативою спадкуванню.
10. **Яке призначення шаблону «Міст» (Bridge)?**
Відокремити абстракцію від її реалізації, щоб вони могли змінюватися незалежно одна від одної.